

A Primer on State Space Models

Dual-Degree Project Seminar-1

Vishwanath Vinod (EE22Boo2)

Guide: Krishna Pilutla



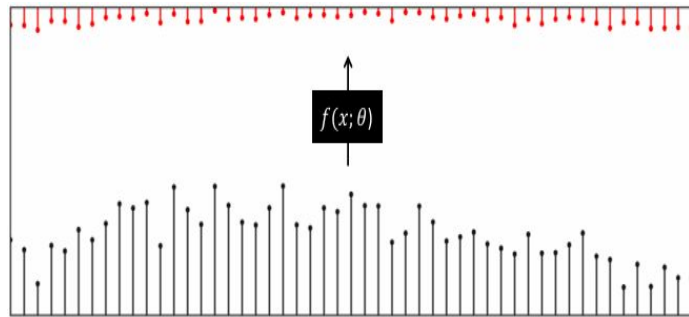
Sequence Models

Parameterized mappings that transform an input sequence into an output sequence.

$$\mathbf{y} = f_{\theta}(\mathbf{x}), \quad [\mathbf{x}_1, \dots, \mathbf{x}_L] \xrightarrow{f_{\theta}} [\mathbf{y}_1, \dots, \mathbf{y}_L].$$

Built on simple computational primitives:

- RNN $\implies$ Recurrence
- CNN $\implies$ Convolutions
- Transformer $\implies$ Attention
- Neural ODE $\implies$ Differential Equations



The inherent limitations of these underlying primitives are passed onto the sequence model,

Limitations of Existing Sequence Models

Convolutions (CNN)

- Fast, parallelizable training
- Expensive inference for long sequences
- Limited context: fixed receptive field

Differential Equations (Neural ODE)

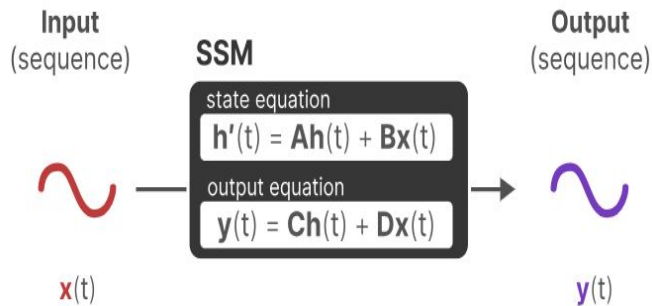
- Continuous-time modeling
- Inefficient: expensive function evaluations
- Stability issues for long sequences

Recurrence (RNN)

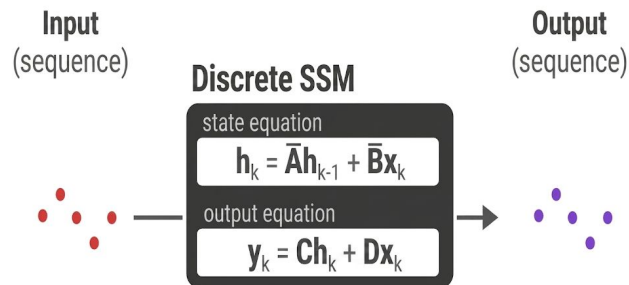
- Fast inference: constant computation per step
- Slow, non-parallelizable training
- Limited context: compressed state space, vanishing gradients

What are State Space Models (SSMs)?

Sequence models that maintain a hidden state evolving according to linear dynamical equations.



SSM for Continuous data



SSM for Discrete data

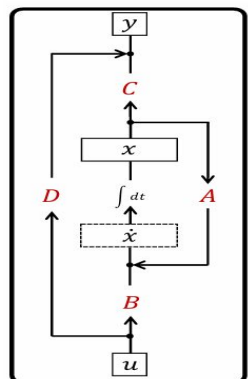
Evolution of SSM research in existing Literature.

[HiPPO](#)¹ $\longrightarrow$ [LSSL](#)² $\longrightarrow$ [S4](#)³, [S4D](#)⁴ $\longrightarrow$ [MAMBA](#)⁵

Complementary Views of SSMs

SSMs combine strengths of **convolution**, **recurrence**, and **continuous** dynamics by offering 3 complementary views.

Continuous View $\xrightarrow{\text{discretize}}$ Recurrent View $\xrightarrow{\text{unroll}}$ Convolutional View

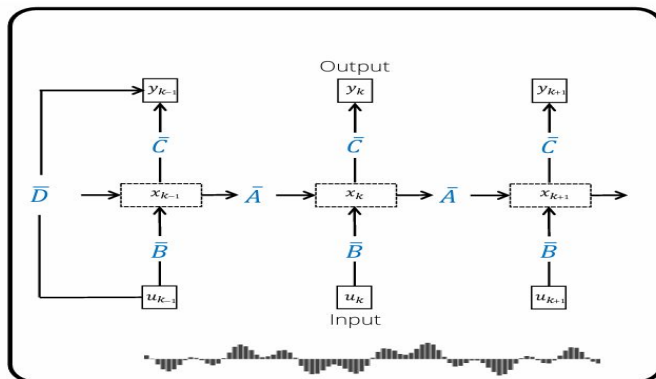


Continuous-time

- ✓ continuous data
- ✓ irregular sampling

Discretize

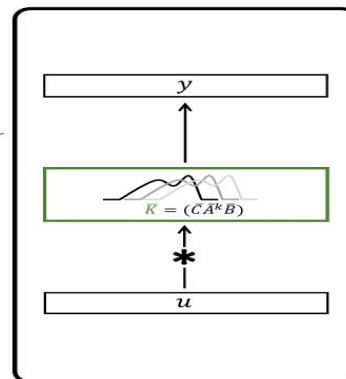
Δt



Recurrent

- ✓ unbounded context
- ✓ efficient inference

or



Convolutional

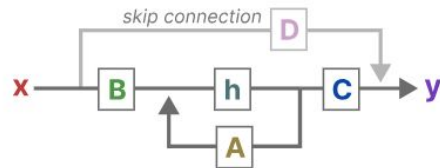
- ✓ local information
- ✓ parallelizable training

Complementary Views of SSMs

Continuous View

$$\dot{h}(t) = Ah(t) + Bx(t), \quad y(t) = Ch(t) + Dx(t).$$

- Simulates continuous processes.
- Provides time-scale adaptation (dependent on inputs).
- Can handle missing data and irregular intervals

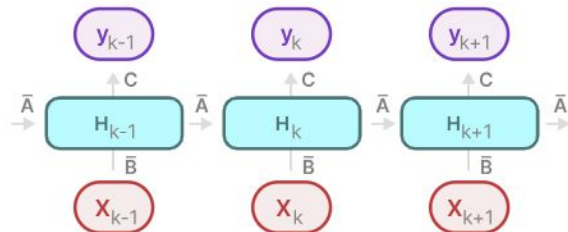


$$A, B \xrightarrow{\text{discretize}} \bar{A}, \bar{B}$$

Recurrent View

$$h_k = \bar{A}h_{k-1} + \bar{B}x_k, \quad y_k = Ch_k + Dx_k.$$

- Recurrent model, with h_k as the hidden state.
- Provides stateful, fast inference.



Complementary Views of SSMs

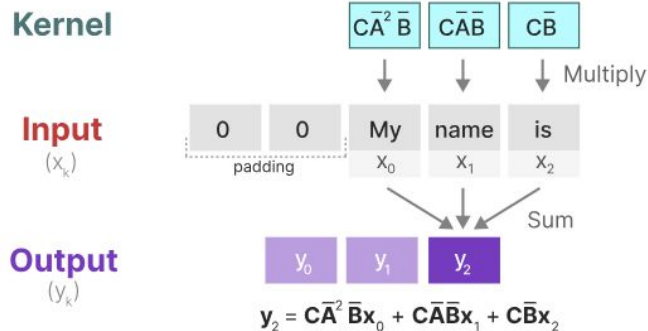
Convolutional View

$$y_k = C\bar{A}^k\bar{B}x_0 + C\bar{A}^{k-1}\bar{B}x_1 + \dots + C\bar{B}x_{k-1} + Dx_k. \quad (\text{assuming } h_{-1} = 0)$$

Krylov Function: $\mathcal{K}_L = (C\bar{B}, C\bar{A}\bar{B}, \dots, C\bar{A}^{L-1}\bar{B})$.

Convolution: $y = \mathcal{K}_L(\bar{A}, \bar{B}, C) * x + Dx$.

- Efficient convolution using FFT in $\mathcal{O}(L \log(L))$.
- Fast, parallelizable training.



SSMs adopt:

Convolutional View during Training.
Recurrent View during Inference.

Why SSMs over Transformers?

Transformers (Attention)	State Space Models
Fast parallelizable Training: $\mathcal{O}(L^2H)$	Fast parallelizable Training: $\tilde{\mathcal{O}}(L+N)$ (Convolutional View)
Full Context (Attention Matrix)	Full Context (HiPPO Initialization of A)
Slow Inferencing: $\mathcal{O}(LH)$ per step (Attention Matrix recomputed at each step)	Fast Inferencing: $\mathcal{O}(NH)$ per step (Recurrent View)
Cant handle varying time steps	Time Scale Adaptation (Continuous time View)
Large Memory Overhead: $\mathcal{O}(L^2)$ (Stores K,V,Q matrices)	Memory Efficient: $\tilde{\mathcal{O}}(L+N)$

L : sequence length H : input dimension N : state dimension $x = [x_1, \dots, x_L]$, $x_k \in \mathbb{R}^H$ $y = [y_1, \dots, y_L]$, $y_k \in \mathbb{R}^H$

Discretization in SSMs

- Δt (step size) is the core parameter for the discretization step in SSMs.
- Length of captured dependencies $\propto \frac{1}{\Delta t}$.
- Learning Δt equivalent to:
 - Controlling effective width of convolution kernel.
 - Gating mechanism of RNNs.
- Discretization Methods used in SSM:
 - Zero Order Hold (ZOH)

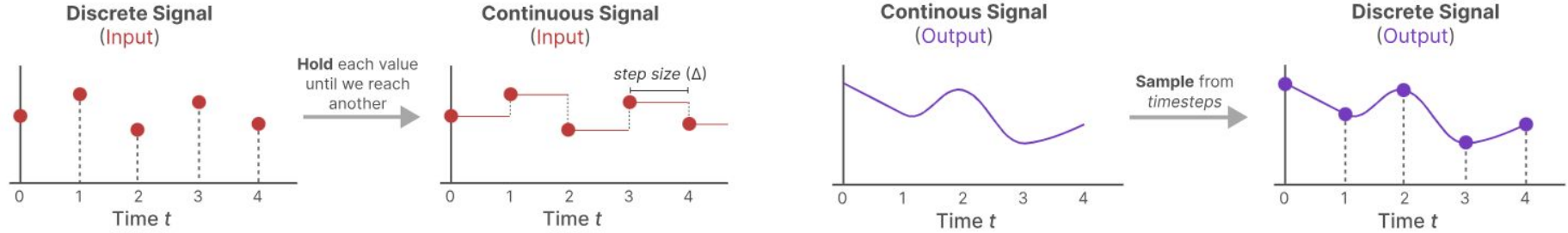
$$\bar{A} = e^{A\Delta t}, \quad \bar{B} = A^{-1}(e^{A\Delta t} - I)B$$

- Generalized Bi-Linear transform (Euler approximation)

$$\begin{aligned}\bar{A} &= (I - \alpha\Delta t A)^{-1}(I + (1 - \alpha)\Delta t A), \\ \bar{B} &= (I - \alpha\Delta t A)^{-1}(\Delta t B).\end{aligned}$$

Discretization in SSMs: ZOH

- The discrete signal is held constant between samples, yielding a continuous-time signal that can be processed by the SSM.



- This can be equivalently represented by setting parameters as:

$$\bar{A} = e^{A\Delta t}, \quad \bar{B} = A^{-1} \left(e^{A\Delta t} - I \right) B$$

and directly operating on the discrete signal

Discretization in SSMs: ZOH

Proving the equivalence of the 2 definitions of ZOH

$$h'(t) = Ah(t) + Bx(t)$$

Homogeneous system:
$$h'(t) = Ah(t) \xrightarrow{\text{solution}} h(t) = e^{A(t-t_0)}h(t_0)$$

Using method of Variation of parameters:
$$h(t) = e^{A(t-t_0)}v(t)$$

Differentiating,
$$\begin{aligned} h'(t) &= Ae^{A(t-t_0)}v(t) + e^{A(t-t_0)}v'(t) \\ &= Ah(t) + e^{A(t-t_0)}v'(t) \end{aligned}$$

$$\implies e^{A(t-t_0)}v'(t) = Bx(t)$$

$$\implies v(t) = v(t_0) + \int_{t_0}^t e^{-A(\tau-t_0)} B x(\tau) d\tau$$

Since $v(t_0) = h(t_0)$, substituting back yields

$$h(t) = e^{A(t-t_0)}h(t_0) + \int_{t_0}^t e^{A(t-\tau)} B x(\tau) d\tau.$$

Discretization in SSMs: ZOH

$$h(t) = e^{A(t-t_0)}h(t_0) + \int_{t_0}^t e^{A(t-\tau)}Bx(\tau) d\tau$$

For $t = (k+1)\Delta t$ and $t_0 = k\Delta t$,

$$x(\tau) = x_k, \quad \forall \tau \in [k\Delta t, (k+1)\Delta t] \quad (\text{ZOH})$$

$$h_{k+1} = e^{A\Delta t}h_k + \left(\int_0^{\Delta t} e^{As} ds \right) Bx_k$$

$$h_{k+1} = e^{A\Delta t}h_k + A^{-1}(e^{A\Delta t} - I)Bx_k.$$

$$\implies \bar{A} = e^{A\Delta t}, \quad \bar{B} = A^{-1}(e^{A\Delta t} - I)B.$$

Hence, we can use the discrete SSM model with these parameters as an alternative way to implement the ZOH discretization.

Discretization in SSMs: GBLT

Applying the Generalized Euler discretization (α) on $h'(t) = Ah(t) + Bx(t)$ gives:

$$\frac{h_{k+1} - h_k}{\Delta t} = A(\alpha h_{k+1} + (1 - \alpha)h_k) + Bx_k.$$

Rearranging gives: $(I - \alpha\Delta t A)h_{k+1} = (I + (1 - \alpha)\Delta t A)h_k + \Delta t Bx_k.$

This is equivalent to a discrete SSM with parameters as:

$$\boxed{\bar{A} = (I - \alpha\Delta t A)^{-1}(I + (1 - \alpha)\Delta t A)} \quad \text{and} \quad \boxed{\bar{B} = (I - \alpha\Delta t A)^{-1}(\Delta t B)}.$$

The trapezoidal ($\alpha = \frac{1}{2}$) scheme is most preferred as it offers $\mathcal{O}(\Delta t^2)$ accuracy.

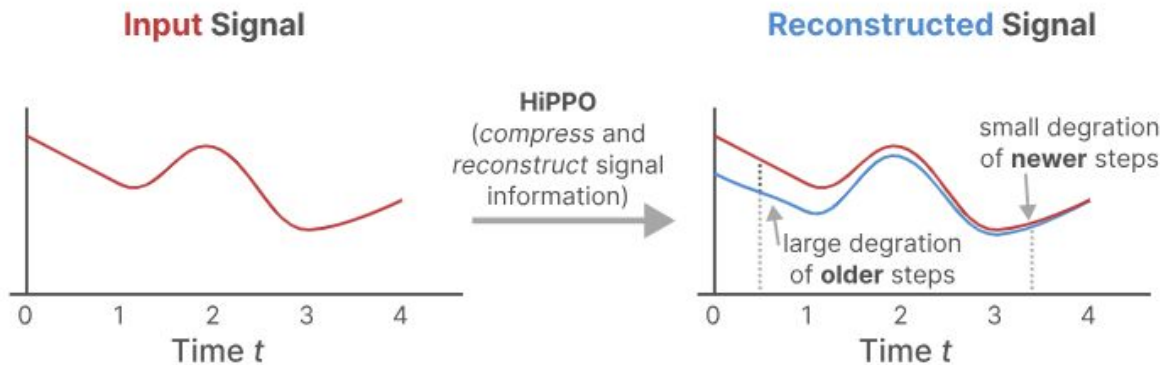
Forward and Backward Euler are the special cases ($\alpha = 0$) and ($\alpha = 1$), respectively.



HiPPO (High Order Polynomial Projection Operators)

How is **A** able to capture long-range dependencies?

- HiPPO (High-order Polynomial Projection Operators) Initialization for **A** .
- Without HiPPO initialization performance of SSMs (**except MAMBA**) decreases significantly.
- HiPPO matrix produces a hidden state that memorizes its history.





HiPPO : Formal Introduction

- **Goal:** Maintain a compressed representation of the entire input history($f_{\leq t} = \{f(x) : x \leq t\}$).
This is an online function approximation problem.



- **Measure:** Weights past information (controls what is remembered) during approximation.
: Defines approximation quality (distance) in function space.

Inner Product: $\langle f, g \rangle_{\mu} = \int_0^{\infty} f(x) g(x) d\mu(x)$ Norm: $\|f\|_{L^2(\mu)} = \langle f, f \rangle_{\mu}^{1/2}$

- Projected history can be represented by the N coefficients of its expansion in any basis in $\mathcal{G}$
- **Basis: Orthogonal Polynomials** (OP) wrt the chosen measure are ideal choice.
: Optimal coefficients can be expressed in closed form.



HiPPO: Prerequisites: Orthogonal Polynomials

Every measure induces a unique sequence of orthogonal polynomial
(which form an orthogonal basis):

$$P_0(x), P_1(x), \dots \quad \text{satisfying} \quad \deg(P_n) = n, \quad \langle P_n, P_m \rangle_\mu = 0 \quad (n \neq m).$$

- **Legendre Polynomial** ($P_n(x)$)

- Orthogonal wrt the measure $1_{[-1,1]}$

$$\frac{2n+1}{2} \int_{-1}^1 P_n(x) P_m(x) dx = \delta_{nm} \quad P_n(1) = 1, \quad P_n(-1) = (-1)^n$$

- For measure $1_{[0,t]}$ scale and shift: $\sqrt{2n+1} P_n\left(\frac{2x}{t} - 1\right)$ (HiPPO-LegS)
- For measure $\frac{1}{\theta} I_{[t-\theta, t]}$ similarly: $\sqrt{2n+1} P_n\left(\frac{2x-t}{\theta} + 1\right)$ (HiPPO-LegT)



HiPPO: Prerequisites: Orthogonal Polynomials

- **Laguerre Polynomial** ($L_n(x)$)

- Standard laguerre is orthogonal wrt the measure $e^{-x}I_{[0,\infty]}$ (HiPPO-LagT).
- Generalized $L_n^{(\alpha)}(x)$ is orthogonal wrt the measure $x^\alpha e^{-x}I_{[0,\infty]}$

$$\int_0^\infty x^\alpha e^{-x} L_n^{(\alpha)}(x) L_m^{(\alpha)}(x) dx = \frac{(n+\alpha)!}{n!} \delta_{nm} \quad L_n^{(\alpha)}(0) = \binom{n+\alpha}{n} = \frac{\Gamma(n+\alpha+1)}{\Gamma(n+1)\Gamma(\alpha+1)}$$

- **Chebyshev Polynomial** ($T_n(x)$)

- Orthogonal wrt the measure $(1-x^2)^{-1/2}I_{(-1,1)}$.
- Not commonly used but can still derive a basis for other measures.



HiPPO : Problem Formulation

HiPPO operator maps an input function to the coefficients of its optimal polynomial approximation, obtained through the composition:

$$\boxed{\text{HiPPO} = \text{coef} \circ \text{proj}, \quad (\text{HiPPO}(f))(t) = \text{coef}_t(\text{proj}_t(f)).}$$
$$f_{\leq t} \xrightarrow{\text{proj}_t} \hat{g}(t) \xrightarrow{\text{coef}_t} \mathbf{c}(t)$$

- **Projection operator:** proj_t

Maps function history to a polynomial $g^{(t)} \in \mathcal{G}$ that minimizes the approximation error.

$$g^{(t)} = \arg \min_{g \in \mathcal{G}} \|f_{\leq t} - g\|_{L^2(\mu(t))}.$$

- **Coefficient operator:** coef_t

Extracts the coefficients of the polynomial $g^{(t)}$ in the OP basis defined wrt the measure $\mu(t)$.

$$c_n(t) = \left\langle g^{(t)}, g_n \right\rangle_{\mu(t)}, \quad \{g_n\}_{n=0}^{N-1} \text{ is an orthogonal polynomial basis w.r.t. } \mu(t).$$

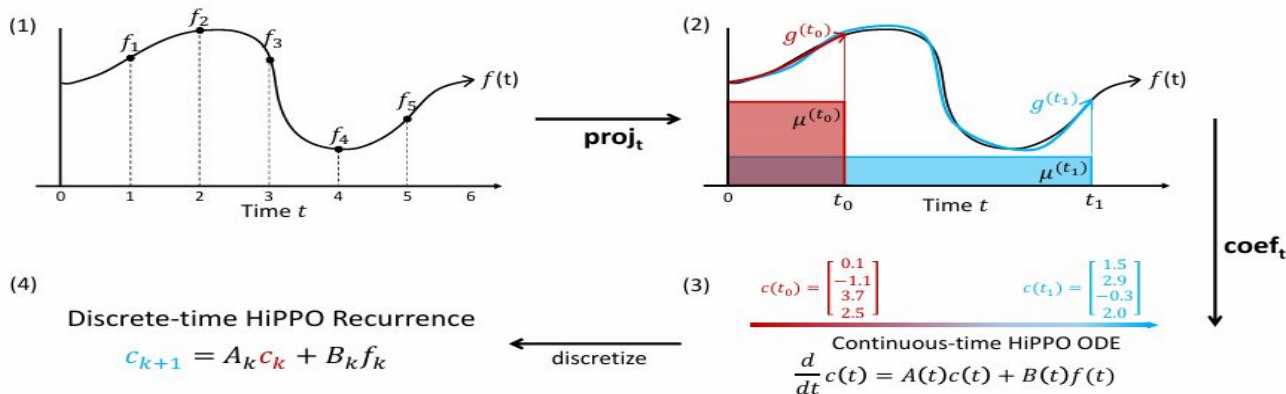
$$\mu(t) : (-\infty, t] \quad f : \mathbb{R}_{\geq 0} \rightarrow \mathbb{R}, \quad \mathbf{c}(t) \in \mathbb{R}^N$$



HiPPO : Coefficient Extraction

Differentiating these coefficients yields an online update rule:

$$\frac{d}{dt}c_n(t) = F_n(c_0(t), \dots, c_{N-1}(t), f(t)) \longrightarrow \dot{\mathbf{c}}(t) = A(t)\mathbf{c}(t) + B(t)f(t) \quad \text{where } A(t) \in \mathbb{R}^{N \times N}, B(t) \in \mathbb{R}^{N \times 1}$$



Orthogonality provides closed-form coefficients;

Differentiating them yields a linear state-space memory update &

Discretizing this dynamics yields an efficient closed-form recurrence for online compression.

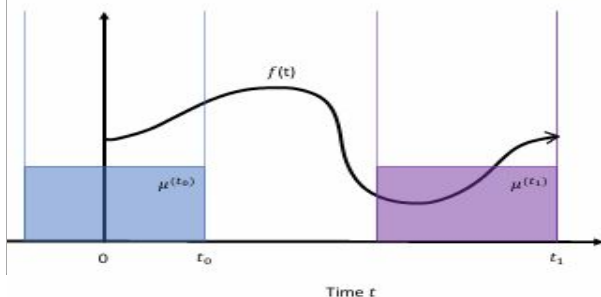


HiPPO : Measure Types

LegT (Translated Legendre)

Uniform weighting over a sliding window of length θ .

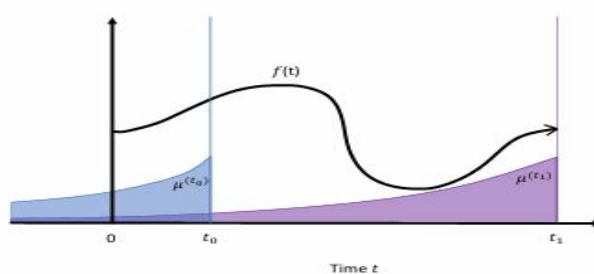
$$\mu^{(t)}(x) = \frac{1}{\theta} \mathbf{1}_{[t-\theta, t]}(x)$$



LagT (Translated Laguerre)

Exponentially decaying weighting over entire history.
Emphasizes recent history

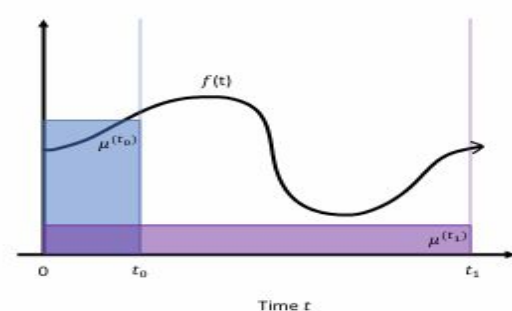
$$\mu^{(t)}(x) = e^{-(t-x)} \mathbf{1}_{(-\infty, t]}(x)$$



LegS (Scaled Legendre)

Uniform weighting over the entire history $[0, t]$.

$$\mu^{(t)}(x) = \frac{1}{t} \mathbf{1}_{[0, t]}(x)$$





HiPPO :Measure Types: Dynamics

Measure Type	Dynamics	Matrix
LegT	$\dot{\mathbf{c}}(t) = -\mathbf{A}\mathbf{c}(t) + \mathbf{B}f(t)$ (LTI dynamics)	$A_{nk} = \frac{1}{\theta} \begin{cases} (-1)^{n-k}(2n+1), & n \geq k, \\ 2n+1, & n < k, \end{cases}$ $B_n = \frac{1}{\theta}(2n+1)(-1)^n$
LagT	$\dot{\mathbf{c}}(t) = -\mathbf{A}\mathbf{c}(t) + \mathbf{B}f(t)$ (LTI dynamics)	$A_{nk} = \begin{cases} 1, & n \geq k, \\ 0, & n < k, \end{cases}$ $B_n = 1$
LegS	$\dot{\mathbf{c}}(t) = -\frac{1}{t}\mathbf{A}\mathbf{c}(t) + \frac{1}{t}\mathbf{B}f(t)$ (Time varying, scale-invariant)	$A_{nk} = \begin{cases} \sqrt{(2n+1)(2k+1)}, & n > k, \\ n+1, & n = k, \\ 0, & n < k, \end{cases}$ $B_n = \sqrt{2n+1}$



HiPPO : Advantages of HiPPO-LegS

- **Timescale Robustness**

- Invariant to input timescale; dilating input doesn't change representation.
- No timescale hyperparameters (no manual tuning required, adaptive)

- **Fast, efficient computation**

- Special structure for discretizing state matrix A.
- Each recurrence step computed in $\mathcal{O}(N)$ (instead of $\mathcal{O}(N^2)$).

- **Bounded Gradients and Approximation errors**

- Solves the Vanishing gradient problem (exponential decay of gradient magnitudes).

$$\forall t_0 < t_1, \quad \left\| \frac{\partial \mathbf{c}(t_1)}{\partial f(t_0)} \right\| = \Theta\left(\frac{1}{t_1}\right) \quad (\text{polynomial decay})$$

- Approximation error decreases with polynomial degree and input smoothness.

LSSL (Linear State Space Layer)

- LSSL was the first SSM which was proposed.
 - Generalizability
 - RNNs and CNNs can be viewed as special cases of LSSLs.
 - Long Range dependencies (LRD)
 - HiPPO initialization of state matrix A takes care of LRD.
 - Computational Complexity
 - Proposed a novel computationally efficient algorithm.
 - Empirical Performance
 - Matches or outperforms SOTA sequence models across various benchmarks.

LSSL: Generalizability: CNNs

All finite length 1D convolutional kernels can be approximated as LSSLs.

Proof Sketch: A 1-D convolution can be written as:

$$y(t) = \int h(\tau) x(t - \tau) d\tau \xrightarrow[\mathcal{L}]{\text{Laplace Transform}} Y(s) = H(s)X(s)$$

If convolution is finite width ($h(t)$ has finite support) then $H(s)$ is analytic.

$$H(s) = \int_0^\infty h(t) e^{-st} dt \sim \int_0^\infty h(t) \sum_{k=0}^\infty \frac{(-st)^k}{k!} dt = \sum_{k=0}^\infty \frac{(-s)^k}{k!} \int_0^\infty t^k h(t) dt.$$

$$\implies H(s) = \sum_{k=0}^\infty c_k s^k, \quad \text{where} \quad c_k = \frac{(-1)^k}{k!} \int_0^\infty t^k h(t) dt.$$

Pade approximation yields an N-degree rational approximation given by:

$$H(s) \approx H_N(s) = \frac{P_N(s)}{Q_N(s)} = \frac{\sum_{k=0}^{N-1} b_k s^k}{s^N + \sum_{k=0}^{N-1} a_k s^k}$$

LSSL: Generalizability:CNNs

Since $Y(s) = H(s)X(s)$:

$$\begin{aligned} (a_0 + \dots + a_{N-1}s^{N-1} + s^N)Y(s) &= (b_0 + b_1s + \dots + b_{N-1}s^{N-1})X(s) \\ \mathcal{L}^{-1} \downarrow \\ y^{(N)}(t) + a_{N-1}y^{(N-1)}(t) + \dots + a_0y(t) &= b_{N-1}x^{(N-1)}(t) + \dots + b_0x(t) \end{aligned}$$

This can be rewritten in controller canonical form which results in state dynamics of LSSL.

$$\dot{\mathbf{h}}(t) = A\mathbf{h}(t) + Bx(t), \quad y(t) = C\mathbf{h}(t), \quad \text{where}$$

$$A = \begin{bmatrix} 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & & & \ddots & \vdots \\ 0 & 0 & 0 & \dots & 1 \\ -a_0 & -a_1 & -a_2 & \dots & -a_{N-1} \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 0 \\ 1 \end{bmatrix}, \quad C = [b_0 \quad b_1 \quad \dots \quad b_{N-1}], \quad \mathbf{h}(t) = [y(t) \quad \dot{y}(t) \quad \dots \quad y^{(N-1)}(t)]^\top$$

LSSL: Generalizability: RNNs

- Same nonlinear dynamical system (underlying nonlinear ODE) can be viewed as either as a RNN or a LSSL.

Single-Layer Nonlinear RNN $\xleftarrow{\text{Time Discretization}}$ Nonlinear Dynamical System $\xrightarrow{\text{Picard Iteration}}$ Deep Linear Continuous-Time LSSL

The key difference lies in where the nonlinearity is expressed/represented:
across **time (RNN)** or across **depth (LSSL)**.

- Gating mechanism in RNNs arise naturally from discretization and are not a heuristic.

Gating Mechanisms $\iff$ Time Discretization

For proving these concepts we will use $\dot{h}(t) = -h(t) + f(t, h(t))$ as the ODE.

LSSL: Generalizability: RNNs

Deep stacked LSSL layers of order $N = 1$ with position-wise nonlinear functions can approximate any nonlinear ODE

Proof Sketch (We use Picard iterations for this, so I will briefly cover it):

Picard-Lindelof Theorem:

- For a non-linear ODE given by $\dot{x}(t) = f(t, x(t))$, $x(t_0) = x_0$ the Picard iterates are:

$$x_{i+1}(t) = x(t_0) + \int_{t_0}^t f(s, x_i(s)) ds. \quad (\textbf{Picard iteration})$$

- Successive Picard iterations $x_0(t), x_1(t), \dots$, converge to the unique solution $x(t)$.

Picard iteration repeatedly applies the integral form of the ODE to construct increasingly accurate approximations of the true solution.

Each layer of a deep LSSL can be viewed as successive Picard iterates.

LSSL: Generalizability: RNNs

Proof Sketch (continued):

We try to approximate the nonlinear ODE $\dot{h}(t) = -h(t) + f(t, h(t))$, for which we take a LSSL with parameters $N = 1$, $A = -1$, $B = C = 1$, $D = 0$.

Each layer of the LSSL implements: $\dot{y}^{(\ell)}(t) = -y^{(\ell)}(t) + x^{(\ell)}(t)$.

To capture non-linearity, we introduce **non-linear functions between each layer (depth)**.

$$x^{(\ell)}(t) = f\left(t, y^{(\ell-1)}(t)\right). \quad \dot{y}^{(\ell)}(t) = -y^{(\ell)}(t) + f\left(t, y^{(\ell-1)}(t)\right).$$

So the ℓ^{th} layer can be viewed as a Picard iteration of the nonlinear ODE, $\dot{h}(t) = -h(t) + f(t, h(t))$.

By the Picard-Lindelof Theorem,

$$y^{(\ell)}(t) \longrightarrow y(t) \quad \text{as } \ell \rightarrow \infty, \quad \text{where } y(t) \text{ is the unique solution of the nonlinear ODE.}$$

$\therefore$ Deep LSSLs can approximate nonlinear ODEs using Picard iteration.

LSSL: Generalizability: RNNs

A single-layer nonlinear RNN can approximate any nonlinear ODE using time discretization,

Proof Sketch:

Discretizing the same nonlinear ODE:

$$\dot{h}(t) = -h(t) + f(t, h(t)) \xrightarrow[\text{discretization}]{\text{Backward Euler}} \frac{h_k - h_{k-1}}{\Delta t_k} = -h_{k-1} + \bar{f}(h_{k-1}, k)$$

Rearrange using $e^{z_k} = \Delta t_k$ and the sigmoid function:

$$h_k = \frac{1}{1 + \Delta t_k} h_{k-1} + \frac{\Delta t_k}{1 + \Delta t_k} \bar{f}(h_{k-1}, k)$$

$$h_k = (1 - \sigma(z_k)) h_{k-1} + \sigma(z_k) \bar{f}(h_{k-1}, k)$$

This a single layer nonlinear RNN where $\bar{f}(\cdot)$ can be instantiated as $\tanh(W h_{k-1} + U x_k)$.

Also, RNN gates emerge naturally from the choice of discretization step Δt_k .

LSSL: Computational Efficiency

- Computational bottlenecks:
 - **Recurrent View:**
 - L Sequential matrix vector multiplications (MVM) with $\bar{A}$.
 - Naive: MVM: $\mathcal{O}(N^2)$, Overall: $\mathcal{O}(N^2L)$.
 - Structured A: MVM: $\mathcal{O}(N)$, Overall: $\mathcal{O}(NL)$.
 - Squaring trick (Parallelizing by precomputing $\bar{A}, \bar{A}^2, \bar{A}^4, \dots$):
 - Depth: $\mathcal{O}(\log L)$ but Memory: $\mathcal{O}(NL)$
 - **Convolutional View:**
 - Computing Krylov Function $\mathcal{K}_L(A, B, C)$.
 - Similar efficiency as it requires computing L matrix powers:

$$\bar{A}, \bar{A}^2, \dots, \bar{A}^L$$

LSSL: Computational Efficiency: Quasi-Separability

Quasi-Separable Matrices:

- Low-rank off diagonal blocks
(k quasi-separable $\implies \text{rank}(\text{off-diagonal block}) \leq k$)
- Memory: $\mathcal{O}(kN)$, MVM complexity: $\mathcal{O}(kN)$.

Restricting A to be quasi-separable resolves the bottlenecks.

HiPPO matrix (with a classical OP basis) are k quasi-separable (with $k \leq 3$).

Ex: HiPPO-LegS: $k=1$.

So, HiPPO initialization handles both LRD and computational efficiency.

LSSL: Computational Efficiency: Algorithm

For a k quasi-separable matrix A , $\mathcal{K}_L(A, B, C)$ can be computed in:

Time: $\tilde{\mathcal{O}}(N + L)$ Depth: $\mathcal{O}(\log L)$ Memory: $\mathcal{O}(N + L)$

Proof Sketch:

- STEP-1: Representing $\mathcal{K}_L(A, B, C)$ in terms of $(I - Ax)^{-1}$ where x is a scalar.
- STEP-2: Compute $(I - Ax)^{-1}$ efficiently using Woodbury Identity and a Divide & Conquer algorithm

STEP-1:

$\mathcal{K}_L(A, B, C)$ is the coefficient vector of the truncated power series:

$$\mathcal{K}_L(A, B, C) = C(I - Ax)^{-1}B \pmod{x^L}$$

Extracting the first L coefficients once $(I - Ax)^{-1}$ is computed can be done in $\mathcal{O}(L \log L)$.

LSSL: Computational Efficiency: Algorithm

STEP-2:

Two important properties are used here:

1. Woodbury Matrix Identity:

Expresses the inverse of $(A + UV^\top)$, in terms of A^{-1} and a small matrix inversion
(Matrix inverse of a low-rank perturbation of another matrix whose inverse is easy to compute.)

$$(A + UV^\top)^{-1} = A^{-1} - A^{-1}U(I + V^\top A^{-1}U)^{-1}V^\top A^{-1},$$

2. Recursive self-similarity of Quasi-separable Matrices:

(Principal submatrices are also quasi-separable).

$$A = \begin{pmatrix} A_{00} & A_{01} \\ A_{10} & A_{11} \end{pmatrix} \text{ is } q\text{-quasiseparable} \implies A_{00} \text{ and } A_{11} \text{ are } q\text{-quasiseparable and } A_{01}, A_{10} \text{ have rank } \leq q.$$

LSSL: Computational Efficiency: Algorithm

STEP-2:

Since A is q -quasi-separable, by recursive self-similarity:

A_{10} and A_{01} can be written as $U_L V_L^T$ and $U_U V_U^T$ where $U_L, V_L, U_U, V_U \in \mathcal{R}^{N \times q}$

This is used to represent $I - Ax$ as low-rank perturbation of a block-diagonal matrix.

$$I - Ax = \begin{bmatrix} I - A_{00}x & 0 \\ 0 & I - A_{11}x \end{bmatrix} + \begin{bmatrix} 0 & U_U \\ U_L & 0 \end{bmatrix} \begin{bmatrix} V_L & 0 \\ 0 & V_U \end{bmatrix}^T x.$$

Applying the Woodbury Identity on this expression gives:

$$C(I - Ax)^{-1}B = M_1 - M_2(I_{2q} + M_3)^{-1}M_4.$$

To compute matrices M_1, M_2, M_3 and M_4 it is sufficient to compute:

$$\begin{bmatrix} C_0 \\ V_L^T \end{bmatrix} (I - A_{00}x)^{-1} [B_0 \quad U_U], \quad \begin{bmatrix} C_1 \\ V_U^T \end{bmatrix} (I - A_{11}x)^{-1} [B_1 \quad U_L],$$

LSSL: Computational Efficiency: Algorithm

where $C = [C_0 \ C_1]$, $B = \begin{bmatrix} B_0 \\ B_1 \end{bmatrix}$, with $(C_i \in \mathbb{R}^{k \times N/2})$ and $(B_i \in \mathbb{R}^{N/2 \times k})$ for $(i \in \{0, 1\})$.

Since $\mathbf{A}_{00}$ and $\mathbf{A}_{11}$ retain the form of $\mathbf{A}$, this yields two recursive calls on matrices of approximately half the size.

This results in a Divide & Conquer algorithm with complexity $\mathcal{O}(q^3 N (\log^4 N))$.

Overall Complexity:

$(\mathbf{I} - \mathbf{A}x)^{-1}$ can be computed in $\tilde{\mathcal{O}}(N)$, and the first L Krylov coefficients can be extracted in $\mathcal{O}(L \log L)$.

$\Rightarrow$ Time/Space: $\tilde{\mathcal{O}}(N + L)$, Depth: $\mathcal{O}(\log L)$ where $(q = \text{const})$.

Practically, we restrict $\mathbf{A}$ to be of a 1-quasi-separable form given by:

$\{P(D + T^{-1})Q\}$ with diagonal D, P, Q and tridiagonal T (satisfied by most HiPPO matrices).

LSSL: Empirical Results

LSSL-fixed / LSSL-f: A fixed as one of the HiPPO matrices (not learnt).

LSSL full / LSSL: A is learnt (initialized with HiPPO matrix).

1. Image & Time-Series Benchmark:

- Sets SOTA on **sCIFAR**, **sMNIST** and **permuted MNIST** with **5×** fewer parameters.
- Reduces RMSE by **2/3** on **BDIMC** healthcare datasets (time series with length $\geq 40,000$) .

2. Extremely Long Signals:

- **Raw speech** (extremely long sequences ($\approx 16,000$) due to high sampling rates):
 - LSSL processes raw speech directly, outperforming models even on **100×** downsampled inputs by over **20%** while being **5×** faster.
- **Sequential CelebA** (flatten **178 × 218** images into **38,000** length sequences):
 - LSSL-f achieves accuracy within **2.16%** of ResNet-18 with **10×** fewer parameters.

LSSL: Empirical Results

3. Ablation Study: $\mathbf{A}$ and Δt

- *Random initialization* for $\mathbf{A}$ (not HiPPO) shows very poor performance.
- Learning Δt adds only $\mathcal{O}(H)$ parameters.
- Learning $\mathbf{A}$ adds $\mathcal{O}(N)$ parameters.
- Performance increases from LSSL-f to LSSL (learnable $\mathbf{A}$).
- Learning Δt generalizes RNN gating.
- It also gives an orthogonal performance boost to learning $\mathbf{A}$.

$\leq 1\%$ parameter count

S4: Structured Sequential SSM

LIMITATIONS OF LSSL:

- The proposed algorithm for computing $\mathcal{K}_L$ is **numerically unstable**.
- Computing $\mathcal{K}_L$ therefore costs $\mathcal{O}(N^2L)$ time and $\mathcal{O}(NL)$ space.
- LSSL additionally requires $\mathcal{O}(NL)$ memory during training to store N -dimensional states.

WHAT S4 DOES?

- S4 resolves this issue by imposing a **special structure on A** .
- The resulting algorithm enables **stable** computation of the kernel optimally in $\tilde{\mathcal{O}}(N + L)$, time using $\tilde{\mathcal{O}}(N + L)$ memory.

S4: DPLR structure for A

Key Idea:

- A has to be a **HiPPO** matrix to satisfy LRD memory retention.
- **Diagonal A** matrix reduces MVM complexity from $\mathcal{O}(N^2)$ to $\mathcal{O}(N)$.

LEMMA: Conjugation Equivalence of SSMS

Two SSMSs are equivalent and induce the same input-output map if related by a change of basis:

$$(A, B, C) \sim (V^{-1}AV, V^{-1}B, CV).$$

Proof:

$$\begin{aligned}\dot{\tilde{h}} &= V^{-1}AV\tilde{h} + V^{-1}Bx, & y &= CV\tilde{h}, \\ (V\dot{\tilde{h}}) &= A(V\tilde{h}) + Bx, & y &= C(V\tilde{h})\end{aligned}$$

They are equivalent SSMSs with a change of basis in state space given by $h = V\tilde{h}$ and produces the same mapping $x \Rightarrow y$.

S4: DPLR structure for A

- By conjugate equivalence of SSMs, we can replace $\mathbf{A}$ with a similar matrix $\mathbf{V}^{-1}\mathbf{A}\mathbf{V}$.
- So we can use the **diagonalized form** of $\mathbf{A}$ for efficient computation.

- HiPPO matrices admit a closed-form diagonalization with $V_{i,j} = \binom{i+j}{i-j}$.
- With this $\mathcal{K}_L$, becomes a **Vandermonde product** computable in $\tilde{\mathcal{O}}(N+L)$.

$$(A, B, C) \sim (\Lambda, \tilde{B}, \tilde{C}), \quad \tilde{B} = V^{-1}B, \quad \tilde{C} = CV, \quad A = V\Lambda V^{-1} \quad \Rightarrow \quad \mathcal{K}_L = \underbrace{\begin{bmatrix} 1 & 1 & \cdots & 1 \\ \lambda_1 & \lambda_2 & \cdots & \lambda_N \\ \lambda_1^2 & \lambda_2^2 & \cdots & \lambda_N^2 \\ \vdots & \vdots & \ddots & \vdots \\ \lambda_1^{L-1} & \lambda_2^{L-1} & \cdots & \lambda_N^{L-1} \end{bmatrix}}_{\text{Vandermonde matrix}} \begin{bmatrix} \tilde{C}_1 \tilde{B}_1 \\ \tilde{C}_2 \tilde{B}_2 \\ \vdots \\ \tilde{C}_N \tilde{B}_N \end{bmatrix}.$$

$$CA^k B = \tilde{C} \Lambda^k \tilde{B} = \sum_{i=1}^N \tilde{C}_i \tilde{B}_i \lambda_i^k.$$

Problem: However, this diagonalization is **numerically unstable**.

Ex.: $V_{3i,i} = \binom{4i}{2i} \approx 2^{4i}$, so entries can grow up to $2^{4N/3}$.

S4: DPLR structure for A

- For numerical stability, V has to be **well conditioned** (unitary) $\implies A$ has to be **Normal**.
- However, HiPPO matrices are not Normal.

All HiPPO matrices are of **NPLR (Normal Plus Low Rank)** form.

$$A = \underbrace{V\Lambda V^*}_{\text{Normal}} - \underbrace{\tilde{P}\tilde{Q}^*}_{\text{Low-rank}},$$
$$\sim \Lambda - PQ^* \quad (\text{Conjugate equivalence}).$$

where V is unitary, Λ is diagonal, and $P, Q \in \mathbb{R}^{N \times r}$ with small rank r .

Diagonalizing HiPPO matrix yields a **DPLR** (Diagonal Plus Low Rank) form for A .

DPLR form improves numerical stability, but doesn't directly give fast computation, unlike diagonal matrices.

S4D: How Diagonal A also works?

An interesting empirical observation was that simply removing the low-rank portion results in a diagonal matrix that performs comparably to the original S4 method.

$$A = V\Lambda V^* - PQ^* = \underbrace{A^{(N)}}_{\text{(Normal-/Skew-HiPPO)}} - \underbrace{PQ^*}_{\text{Low-rank correction}}$$

S4 uses the full NPLR matrix , while S4D retains only the diagonalized skew-HiPPO component:

$$A^{(D)} = V^* A^{(N)} V = \Lambda.$$

Method	Structure	Kernel Computation	Discretization	Constraint $\Re(\mathbf{A})$	Trainable $\mathbf{B}$	Initialization of $\mathbf{A}$
S4	DPLR	Cauchy	Bilinear	exp	Yes	HiPPO
DSS	diagonal	softmax	ZOH	id (none)	No	HiPPO-D
S4D	diagonal	Vandermonde	either	exp / ReLU	optional	various

DSS (Diagonal State Space) is S4D with a fixed B parameter (non-trainable).

S4: Computational Complexity

Recurrence Complexity:

- DPLR structure of $\mathbf{A}$ directly enables optimal recurrence computation.
- For any Δt , one recurrence step can be computed in $\mathcal{O}(N)$ operations.

Convolutional Complexity:

- For any Δt , $\mathbf{K}_L$ can be computed using 4 Cauchy multiplications, requiring only $\tilde{\mathcal{O}}(N + L)$ operations and $\tilde{\mathcal{O}}(N_{\text{space}})$ space.

This is done using a **3-step algorithm**, which uses the DPLR form of $\mathbf{A}$.

S4: Recurrence Computational Complexity

Substitute the DPLR form of A into the Trapezoidal discretization.

$$\text{GBLT } (\alpha = 0.5) : \quad \bar{A} = (I - \frac{\Delta t}{2} A)^{-1} (I + \frac{\Delta t}{2} A), \quad \bar{B} = (I - \frac{\Delta t}{2} A)^{-1} \Delta t B.$$

$$\begin{aligned} (I - \frac{\Delta t}{2} A)^{-1} &= \frac{2}{\Delta t} \left(\frac{2}{\Delta t} I - \Lambda + PQ^* \right)^{-1} & I + \frac{\Delta t}{2} A &= \frac{\Delta t}{2} \left(\frac{2}{\Delta t} I + \Lambda - PQ^* \right) \\ &= \frac{2}{\Delta t} \left(D - DP(I + Q^* DP)^{-1} Q^* D \right) \quad (\text{Woodbury Identity}) & &\triangleq \frac{\Delta t}{2} A_0. \\ &\triangleq \frac{2}{\Delta t} A_1. \end{aligned}$$

Here, A_0 and A_1 are DPLR matrices defined for notational ease and $D = (\frac{2}{\Delta t} - \Lambda)^{-1}$ is a diagonal matrix.

$$\implies \bar{A} = A_1 A_0, \quad \bar{B} = 2A_1 B.$$

Because A_0 and A_1 are DPLR, MVMs cost only $\mathcal{O}(N)$.

$\implies$ Each recurrence step takes $\mathcal{O}(N)$ operations.

S4: Convolutional Computational Complexity

Step 1: Generating Function

- Instead of computing powers of $\mathbf{A}$, use the truncated generating function (SSMGF).

Step 2: Woodbury Identity (Simplifying SSMGF)

- Reduces inverse of a DPLR matrix to a diagonal inverse plus a low-rank correction.

$$(D + PQ^*)^{-1} = D^{-1} - D^{-1}P(I + Q^*D^{-1}P)^{-1}Q^*D^{-1}, \quad \text{where } D \text{ is diagonal.}$$

Step 3: Cauchy Kernel (Simplifying SSMGF)

- Reduce the remaining computation to evaluating multiple cauchy kernels.
- Once SSMGF is computed, evaluate it at roots of unity and recover $\mathcal{K}_L$ using inverse FFT.

S4: Convolutional Computational Complexity

Step 1: Generating Function

- SSM Generating Function (SSMGF)

$$\hat{\mathcal{K}}(z; \bar{A}, \bar{B}, C) = \sum_{i=0}^{\infty} C \bar{A}^i \bar{B} z^i = C(I - \bar{A}z)^{-1} \bar{B}$$

- Truncated SSMGF at node z:

$$\begin{aligned} \hat{\mathcal{K}}_L(z) &= \sum_{i=0}^{L-1} C \bar{A}^i \bar{B} z^i \\ &= C \left(\sum_{i=0}^{L-1} (\bar{A}z)^i \right) \bar{B} \\ &= C(I - \bar{A}^L z^L)(I - \bar{A}z)^{-1} \bar{B} \\ &= \tilde{C}(I - \bar{A}z)^{-1} \bar{B}, \end{aligned}$$

- DPLR matrices:

- Powering : Inefficient (Repeated multiplication destroys DPLR structure)
- Inversion : Efficient (Woodbury identity preserves it)

$$\implies \hat{\mathcal{K}}_L(z) = \tilde{C}(I - \bar{A}z)^{-1} \bar{B}$$

where $\tilde{C} = C(I - \bar{A}^L z^L)$.

S4: Convolutional Computational Complexity

Step 2: Woodbury Identity:

- Substituting the discretized forms in the truncated generating function:

$$\text{GBLT } (\alpha = 0.5) : \quad \bar{A} = (I - \frac{\Delta t}{2} A)^{-1} (I + \frac{\Delta t}{2} A), \quad \bar{B} = (I - \frac{\Delta t}{2} A)^{-1} \Delta t B.$$

$$\begin{aligned} \hat{\mathcal{K}}_L(z) &= \tilde{C} (I - \bar{A} z)^{-1} \bar{B} \\ &= \frac{2}{1+z} \tilde{C} \left(\frac{2}{\Delta} \frac{1+z}{1-z} - A \right)^{-1} B \end{aligned}$$

- Using DPLR structure($A = \Lambda - PQ^*$):

$$\begin{aligned} \hat{\mathcal{K}}_L(z) &= \frac{2}{1+z} \tilde{C} (D(z) + PQ^*)^{-1} B, \quad \text{where } D(z) = \frac{2}{\Delta} \frac{1+z}{1-z} - \Lambda \quad (\text{diagonal}) \\ &= \frac{2}{1+z} \left[\tilde{C} D(z)^{-1} B - \tilde{C} D(z)^{-1} P (I + Q^* D(z)^{-1} P)^{-1} Q^* D(z)^{-1} B \right] \quad (\text{Woodbury identity}). \end{aligned}$$

S4: Convolutional Computational Complexity

Step 3: Cauchy Kernel Multiplication

- The truncated SSMGF can be written as:

$$\hat{\mathcal{K}}_L(z) = \sum_{k=0}^{L-1} \mathcal{K}_k z^k, \quad \text{where } \mathcal{K}_k = C \bar{A}^k \bar{B} \text{ are coefficients of convolution kernel. } (\mathcal{K}_L)$$

- Evaluating at roots of unity ($\Omega = \{\omega_j = e^{-i \cdot (2\pi \frac{j}{L})} \mid j \in [1, \dots, L]\}$):

$$\begin{aligned} \hat{\mathcal{K}}_L(\omega_j) &= \sum_{k=0}^{L-1} \mathcal{K}_k \omega_j^k \\ &= \sum_{k=0}^{L-1} \mathcal{K}_k e^{-i2\pi kj/L} \quad (\text{Discrete Fourier Transform of a length (L) sequence}) \end{aligned}$$

The truncated SSMGF at each root of unity produces exactly the DFT coefficients of the $\mathcal{K}_L$.

S4: Convolutional Computational Complexity

Step 3: Cauchy Kernel Multiplication

- So applying DFT on coefficients of $\hat{\mathcal{K}}_L$ is equivalent to evaluating the truncated SSMGF at roots of unity.

$$\hat{\mathcal{K}} = \begin{bmatrix} \hat{\mathcal{K}}_L(\omega_0) \\ \hat{\mathcal{K}}_L(\omega_1) \\ \vdots \\ \hat{\mathcal{K}}_L(\omega_{L-1}) \end{bmatrix} = \mathcal{F}_L \begin{bmatrix} \mathcal{K}_0 \\ \mathcal{K}_1 \\ \vdots \\ \mathcal{K}_{L-1} \end{bmatrix} = \mathcal{F}_L \mathcal{K}, \text{ where } \mathcal{F}_L \text{ denotes the } L \times L \text{ DFT matrix}$$

- This implies that applying **inverse FFT** we can recover coefficients of $\mathcal{K}_L$ from the values of truncated SSMGF at roots of unity in $\mathcal{O}(L \log L)$ time.

$$\mathcal{K} = \mathcal{F}_L^{-1} \hat{\mathcal{K}} \quad \text{and} \quad \text{Convolutional kernel: } \mathcal{K}_L = [\mathcal{K}_0, \dots, \mathcal{K}_{L-1}]$$

S4: Convolutional Computational Complexity

Step 3: Cauchy Kernel Multiplication

After step 2, all computations reduce to MVM with a diagonal matrix whose entries are Cauchy kernel evaluations.

Cauchy Kernel on nodes $\mathbf{x} = (x_i)_{i=0}^{L-1} \in \mathbb{C}^L$ and $\Lambda = (\lambda_j)_{j=1}^N \in \mathbb{C}^N$:

$$\mathcal{C} = \mathcal{C}(\mathbf{x}, \Lambda) \in \mathbb{C}^{L \times N}, \quad \text{where } \mathcal{C}_{i,j} = \frac{1}{x_i - \lambda_j}$$

The Woodbury Identity yields:

$$\hat{\mathcal{K}}_L(z) = \frac{2}{1+z} \left(\tilde{C}D(z)^{-1}B - \tilde{C}D(z)^{-1}P(I + Q^*D(z)^{-1}P)^{-1}Q^*D(z)^{-1}B \right), \quad \text{where } D(z) = \left(\frac{2}{\Delta t} \frac{1-z}{1+z} I - \Lambda \right).$$

Evaluating this at roots of unity and defining $x_i = \frac{2}{\Delta t} \frac{1 - \omega_i}{1 + \omega_i}$ we can rewrite as:

S4: Convolutional Computational Complexity

Step 3: Cauchy Kernel Multiplication

$$\begin{aligned} D(\omega_i)^{-1} &= \left(\text{diag}(x_i - \lambda_1, \dots, x_i - \lambda_N) \right)^{-1} \\ &= \text{diag}\left(\frac{1}{x_i - \lambda_1}, \dots, \frac{1}{x_i - \lambda_N} \right) \\ &= \text{diag}(\mathcal{C}(x_i, \lambda_1), \dots, \mathcal{C}(x_i, \lambda_N)). \end{aligned}$$

Thus, each diagonal entry is a Cauchy kernel evaluation.

Thus the entire simplification reduces to 4 Cauchy kernel MVMs.

Using Fast Multipole Method (FMM) this can be achieved in $\tilde{\mathcal{O}}(L + N)$ space and time.

S4: Practical Details

NUMERICAL INSTABILITY:

- **Eigenvalues of A on the right half-plane** cause exponentially growing state dynamics, resulting in numerical instability during kernel computation.
- Change the NPLR parametrization: $\Lambda - PQ^* \rightarrow \Lambda - PP^*$.
- Since PP^* is **p.s.d** subtracting by it shifts the eigenvalues leftward.

ABLATION STUDY (on sequential CIFAR-10):

- How important is HiPPO initialization?
 - Large generalization gap:
HiPPO outperforms random initialization by over 15% in validation accuracy.
 - Replacing HiPPO with an arbitrary NPLR parameterization doesn't work either.
- Is training SSM parameters required?
 - Training improved all methods, particularly the randomly initialized ones.

S4: Empirical Results

- **Efficiency:**
 - S4's speed and memory use is competitive with the most efficient Transformer variants (Linear Transformer, Performer) and better than LSSL.
- **Long Range Dependencies (LRD):**
 - **Path-X:**

S4 solves this extremely challenging task (length: $128 \times 128 = 16384$)
 - **Long-Range Arena (LRA):**

S4 advances the SoTA (averaging 80.48%), outperforming all baselines on all tasks (< 60%).
 - **Raw Speech:**

S4 achieves **98.3%** accuracy, higher than all baselines that use the **100×** shorter pre-processed features.

S4: Empirical Results

- **Autoregressive performance:**

- **CIFAR density estimation** (Flattened images, a sequence of 3072 RGB subpixels, predicted one by one):

With no 2-D inductive bias, S4 is performing among the best.

- **WikiText-103 (Language modelling):**

S4 comparable to transformer and sets SOTA for attention-free models.

- **60×** Faster inferencing than vanilla transformer on both tasks.

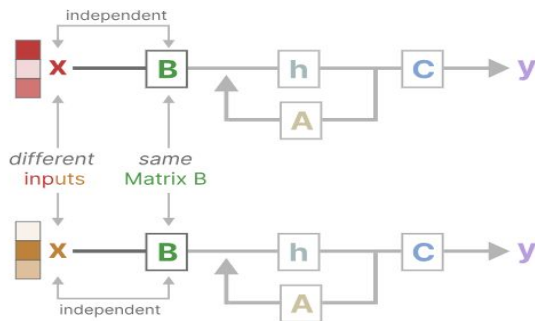
- **Image Classification:**

- Sets SOTA for **sequential-MNIST** and **sequential-CIFAR** tasks, with performance comparable to LSSL (SOTA) in **permuted-MNIST**.

MAMBA

Problems of SSMs upto now:

- Successful in **continuous signal domains** (audio, vision) but not for **discrete data** (text).
- Lack input-dependent selective processing (i.e. focus on or ignore particular inputs).



Constant regardless of the input

$$h_k = \bar{A} h_{k-1} + \bar{B} x_k$$
$$y_k = C h_k$$

What MAMBA does?

- Input-dependent parameters: **Selectivity**.
- Resulting time-varying SSM cannot use convolutions, so proposed a **Hardware aware algorithm** using a **parallel scan** operation for efficient computation.

MAMBA: Selectivity

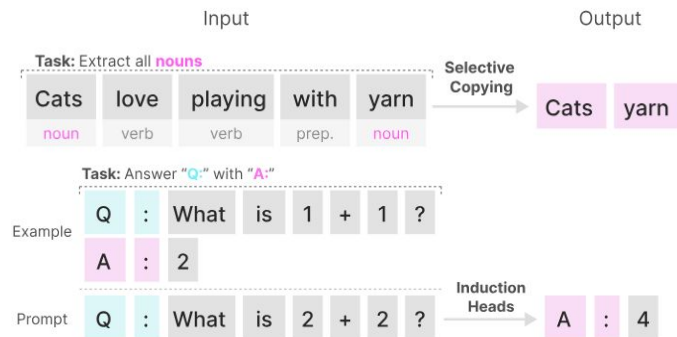
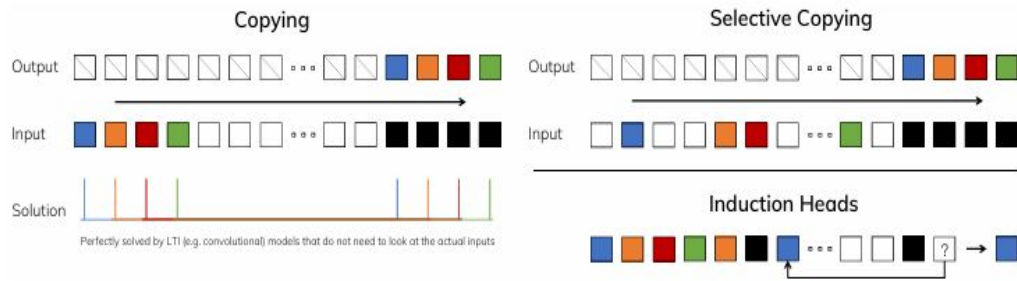
Motivation:

- **Selective Copying:**

Content-aware reasoning to memorize relevant tokens and filter out the irrelevant ones.

- **Induction Heads:**

Context-aware reasoning to produce the correct output in the appropriate context.



Remove the LTI constraint by introducing input-dependent parameters to solve these.

MAMBA: Selectivity: Input-Dependent Parameters

Parameters B, C and Δt are made functions of the input.

$$x \in \mathbb{R}^{B \times L \times D},$$

$$B = \text{Linear}_N(x), \quad B \in \mathbb{R}^{B \times L \times N},$$

$$C = \text{Linear}_N(x), \quad C \in \mathbb{R}^{B \times L \times N},$$

$$\Delta t = \tau_\Delta(\text{Bias} + s_\Delta(x)), \quad \Delta t \in \mathbb{R}^{B \times L \times D},$$

Linear_d : Projection to dimension d ,

$$s_\Delta(x) = \text{Broadcast}_D(\text{Linear}_1(x)) \quad \text{or} \quad \text{Linear}_D(\text{Linear}_R(x)),$$

$$\tau_\Delta(x) = \text{Softplus}(x) = \ln(1 + e^x), \text{ (a smooth, differentiable approximation of ReLU).}$$

The Bias parameter is initialized as $\tau_\Delta^{-1}(\text{Uniform}[\Delta t_{\min}, \Delta t_{\max}])$.

A, D are fixed, while input-dependent Δt induces selectivity in $\bar{A}$ and $\bar{B}$.

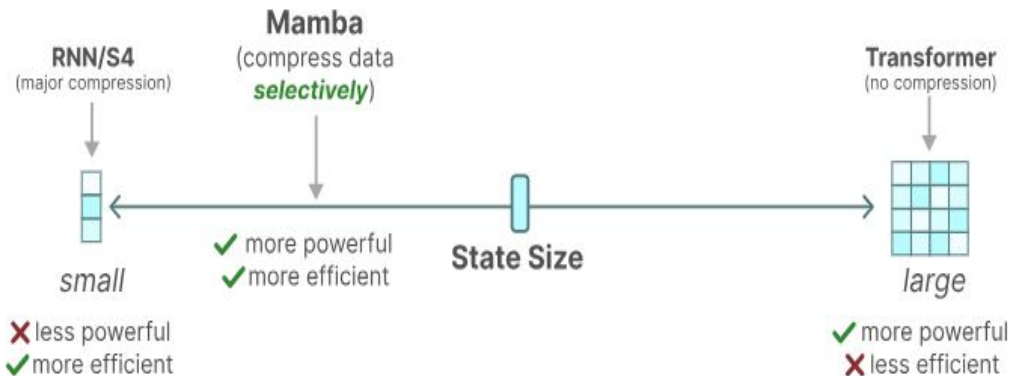
MAMBA: Selectivity: Input-Dependent Parameters

- Δt controls how much to focus or ignore the current input.
 - Large Δt : Resets state and focuses on current input.
 - Small Δt : Ignores current input.
- $s_{\Delta}(x) = \text{Broadcast}_D(\text{Linear}_1(x))$, ensures that:
 - A single scalar Δt is predicted for each token and broadcast to all D channels.
 - A token is either **processed** or **ignored** consistently across all channels (**limited expressivity**).
- $s_{\Delta}(x) = \text{Linear}_D(\text{Linear}_R(x))$, where, R is a small fraction of D .
 - Replace broadcasting with a trainable low-rank projection (R).
 - **Increases expressiveness** while remaining **parameter-efficient**.

MAMBA: Selectivity

Efficiency vs Effectiveness tradeoff:

- Characterized by how well they compress their state.
- **Efficient** models: State must be small.
- **Effective** models: State contains all necessary information from the context.
- **Solution:** Selectivity, context-aware retention of relevant inputs (info).



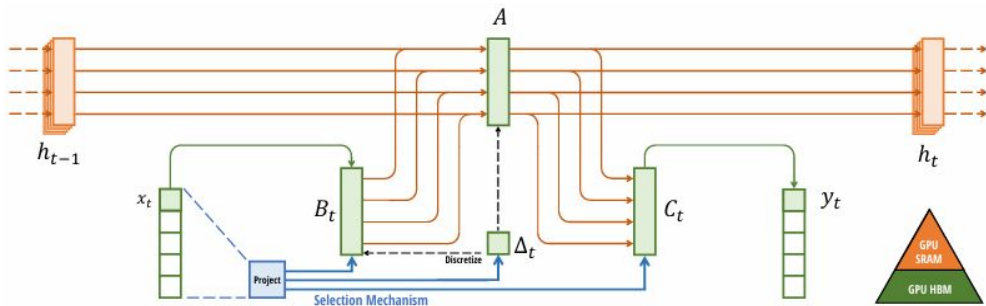
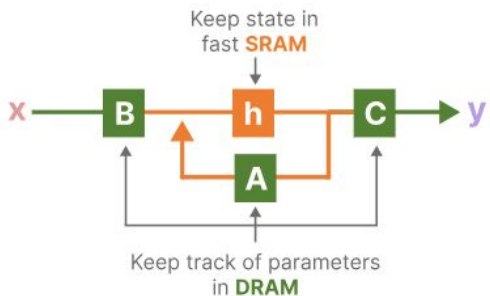
MAMBA: Hardware-aware parallel scan

Computational Bottleneck:

Since SSM is applied independently to each input channel, for $\mathbf{x} \in \mathbb{R}^{B \times L \times D}$:

Computing state $\mathbf{h}$: $\mathcal{O}(BLDN)$ time and memory.

Solution: **Materialize the state $\mathbf{h}$ only in more efficient levels of the memory hierarchy and use a parallelized selective scan for recurrence.**



Algorithm:

Computes the SSM recurrently using a hardware-aware parallelized selective scan, keeping intermediate states on-chip to avoid expensive GPU memory transfers.

MAMBA: Hardware-aware parallel scan

This is achieved using 3 classical techniques: **Kernel Fusion**, **Parallel Scan**, and **Recomputation**

Kernel Fusion: (Handles memory)

- Load the **tiny SSM parameters** once from slow HBM to fast SRAM.

$$A \in \mathbb{R}^{N \times N}, \Delta t \in \mathbb{R}^{B \times L \times D}, C \text{ and } B \in \mathbb{R}^{B \times L \times N}$$

$$\text{But } \bar{A}, \bar{B} \in \mathbb{R}^{B \times L \times N \times D}$$



Frequently copying information between SRAM and DRAM becomes a bottleneck.

- Following are fused into one kernel: **Discretization**, **Selective Scan**, **Multiplication with C**.
 - Perform these inside SRAM and only write back final result to HBM.

MAMBA: Hardware-aware parallel scan

Parallel Blelloch Scan (Handles time complexity):

- Computes all recurrent states in parallel using an **associative** composition.

$$h_k = A_k h_{k-1} + B_k, \xrightarrow{\text{Associative composition}} (A_r, B_r) \circ (A_l, B_l) = (A_r A_l, A_r B_l + B_r).$$

- **Upsweep:** Build a binary tree of partial compositions.
- **Downsweep:** Propagate prefix compositions to every node, giving prefix for each timestep.

$(A_{\text{pre}}^{(t)}, B_{\text{pre}}^{(t)})$: Composition of all operations before timestep t

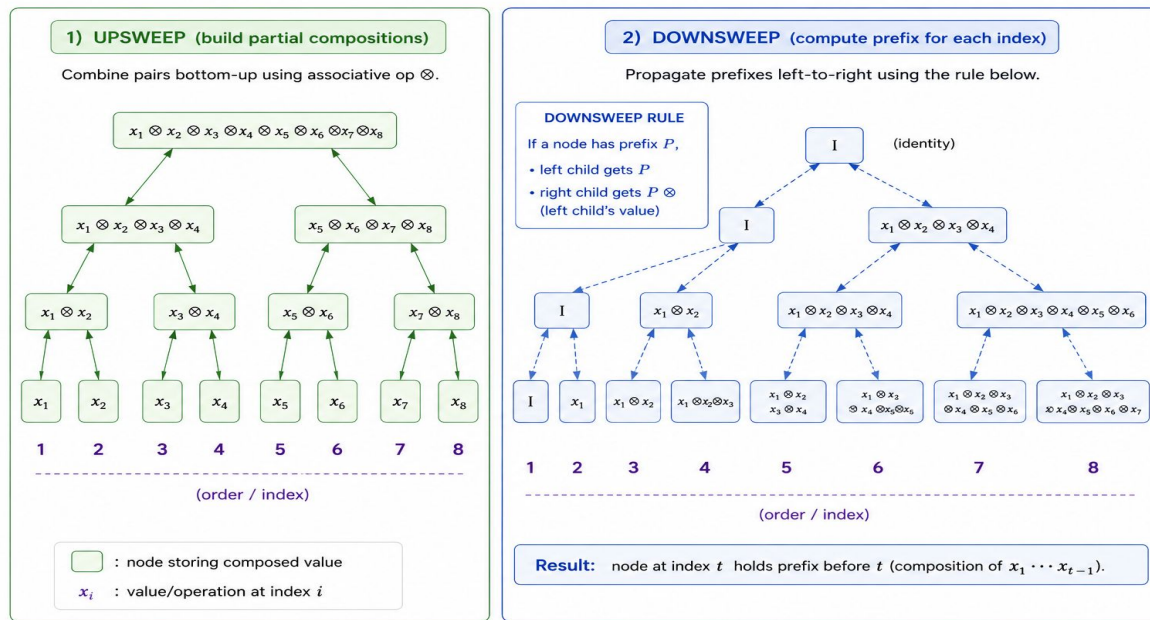
- Compose prefix operator with current operation to obtain state:

$$h_t = \left((A_{\text{pre}}^{(t)}, B_{\text{pre}}^{(t)}) \circ (A_t, B_t) \right)_B, \text{ since } h_{-1} = 0 \text{ and } (\cdot)_B \text{ denotes the second component of the composition.}$$

$$\begin{aligned} \text{EX: } h_0 &= A_0 h_{-1} + B_0 = B_0, & (h_{-1} = 0), \\ h_1 &= A_1 h_0 + B_1 = A_1 B_0 + B_1, \\ &= \left((A_1, B_1) \circ (A_0, B_0) \right)_B. \end{aligned}$$

$\text{Parallel scan (with } L \text{ processors)} = \mathcal{O}(\log L), \quad \text{Recurrence} = \mathcal{O}(L).$
--

MAMBA: Hardware-aware parallel scan

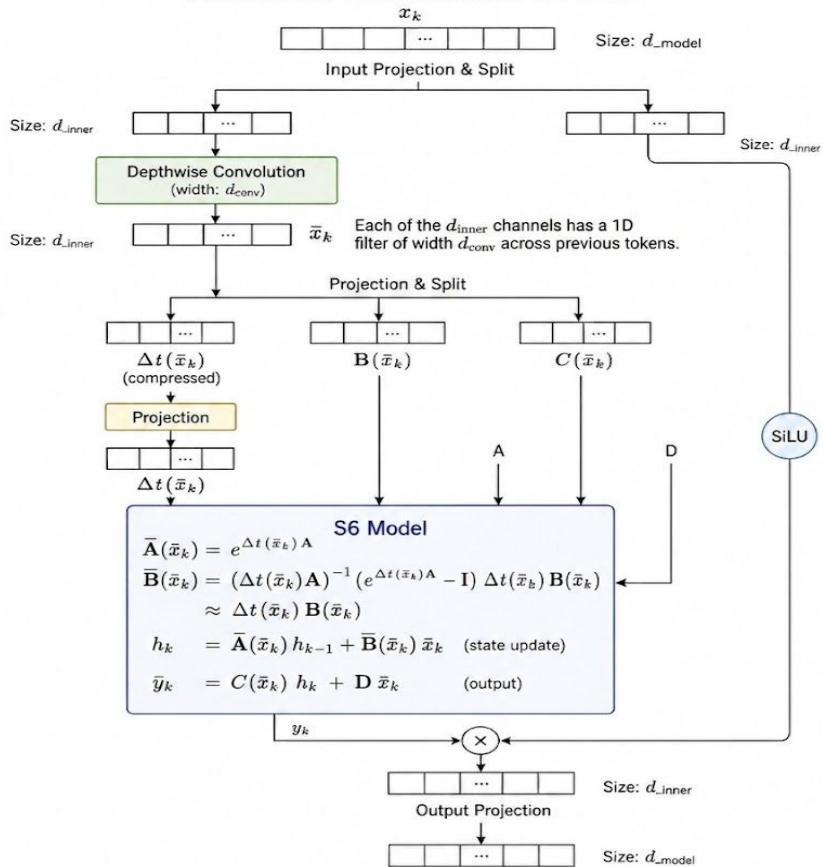


Recomputation:

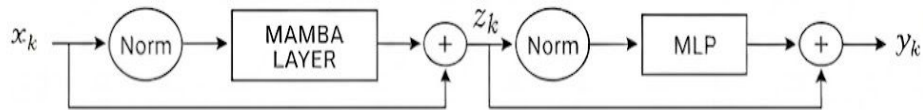
- During backpropagation, you need intermediate states.
- So instead of storing them recompute in the backward pass when the inputs are loaded from HBM to SRAM.

MAMBA: Architecture

ARCHITECTURE: MAMBA LAYER



ARCHITECTURE: MAMBA BLOCK



	Complex State	Real State
Preferred for	Continuous signals	Discrete data
Initialization of A	S4D-Lin	S4D-Real
	$A_n = -\frac{1}{2} + ni$	$A_n = -(n + 1)$

MAMBA: Empirical Results

Synthetic Tasks:

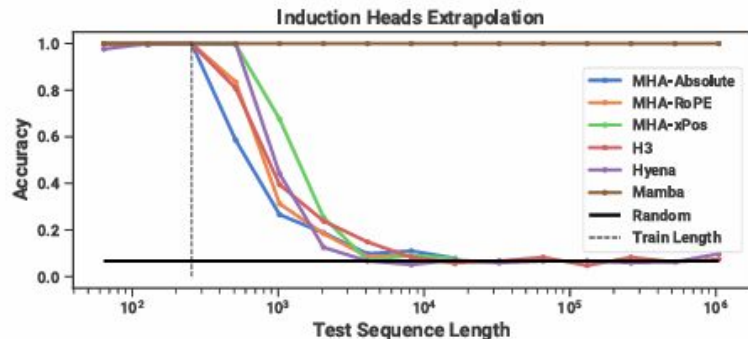
- **Selective Copying:**

- Vanilla case: handled by LTI SSMs (keeps track of time instead of learning from data)
- Selective copying prevents this shortcut by randomizing the spacing between tokens.

- **Induction Heads:**

- Good test for in-context learning ability of LLMs (associative recall and copy).

MODEL	ARCH.	LAYER	Acc.
S4	No gate	S4	18.3
-	No gate	S6	97.0
H3	H3	S4	57.0
Hyena	H3	Hyena	30.1
-	H3	S6	99.7
-	Mamba	S4	56.4
-	Mamba	Hyena	28.4
Mamba	Mamba	S6	99.8

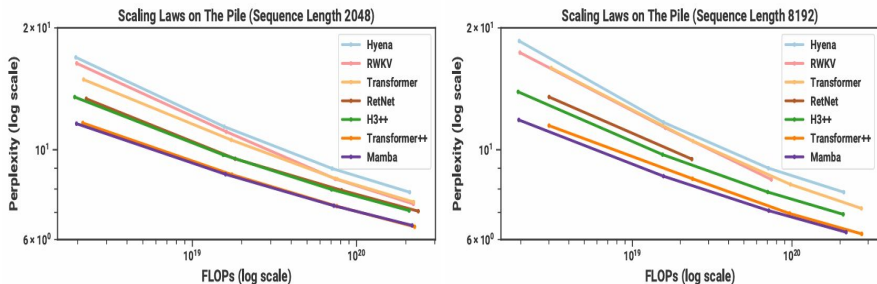


It generalizes perfectly to million-length sequences, or $4000\times$ longer than it saw during training ($L = 256$, $\text{vocab_size}=16$), while no other method goes beyond $2\times$.

MAMBA: Empirical Results

Language Modelling:

First attention-free model to match the performance of a very strong Transformer recipe (Transformer++), particularly as the sequence length grows.



Models of size $\approx 125M$ to $\approx 1.3B$ parameters, trained on the Pile.

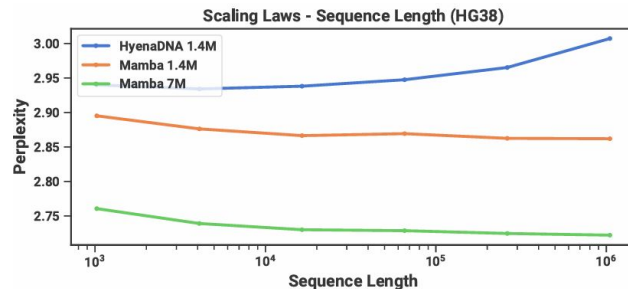
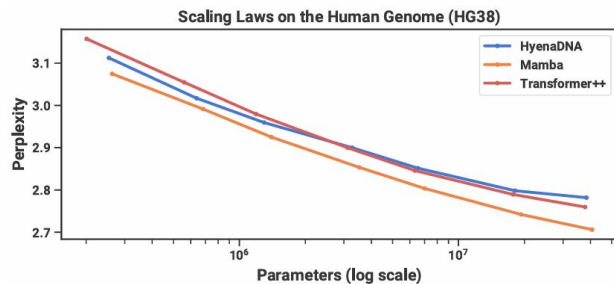
MODEL	TOKEN.	PILE PPL ↓	LAMBADA PPL ↓	LAMBADA ACC ↑	HELLASWAG ACC ↑	PIQA ACC ↑	ARC-E ACC ↑	ARC-C ACC ↑	WINOGRANDE ACC ↑	AVERAGE ACC ↑
Hybrid H3-130M	GPT2	—	89.48	25.77	31.7	64.2	44.4	24.2	50.6	40.1
Pythia-160M	NeoX	29.64	38.10	33.0	30.2	61.4	43.2	24.1	51.9	40.6
Mamba-130M	NeoX	10.56	16.07	44.3	35.3	64.5	48.0	24.3	51.9	44.7
Hybrid H3-360M	GPT2	—	12.58	48.0	41.5	68.1	51.4	24.7	54.1	48.0
Pythia-410M	NeoX	9.95	10.84	51.4	40.6	66.9	52.1	24.6	53.8	48.2
Mamba-370M	NeoX	8.28	8.14	55.6	46.5	69.5	55.1	28.0	55.3	50.0
Pythia-1B	NeoX	7.82	7.92	56.1	47.2	70.7	57.0	27.1	53.5	51.9
Mamba-790M	NeoX	7.33	6.02	62.7	55.1	72.1	61.2	29.5	56.1	57.1
GPT-Neo 1.3B	GPT2	—	7.50	57.2	48.9	71.1	56.2	25.9	54.9	52.4
Hybrid H3-1.3B	GPT2	—	11.25	49.6	52.6	71.3	59.2	28.1	56.9	53.0
OPT-1.3B	OPT	—	6.64	58.0	53.7	72.4	56.7	29.6	59.5	55.0
Pythia-1.4B	NeoX	7.51	6.08	61.7	52.1	71.0	60.5	28.5	57.2	55.2
RWKV-1.5B	NeoX	7.70	7.04	56.4	52.5	72.4	60.5	29.4	54.6	54.3
Mamba-1.4B	NeoX	6.80	5.04	64.9	59.1	74.2	65.5	32.8	61.5	59.7
GPT-Neo 2.7B	GPT2	—	5.63	62.2	55.8	72.1	61.1	30.2	57.6	56.5
Hybrid H3-2.7B	GPT2	—	7.92	55.7	59.7	73.3	65.6	32.3	61.4	58.0
OPT-2.7B	OPT	—	5.12	63.6	60.6	74.8	60.8	31.3	61.0	58.7
Pythia-2.8B	NeoX	6.73	5.04	64.7	59.3	74.0	64.1	32.9	59.7	59.1
RWKV-3B	NeoX	7.00	5.24	63.9	59.6	73.7	67.8	33.1	59.6	59.6
Mamba-2.8B	NeoX	6.22	4.23	69.2	66.1	75.2	69.7	36.3	63.5	63.3
GPT-J-6B	GPT2	—	4.10	68.3	66.3	75.4	67.0	36.6	64.1	63.0
OPT-6.7B	OPT	—	4.25	67.7	67.2	76.3	65.6	34.9	65.5	62.9
Pythia-6.9B	NeoX	6.51	4.45	67.1	64.0	75.2	67.3	35.5	61.3	61.7
RWKV-7.4B	NeoX	6.31	4.38	67.2	65.5	76.1	67.8	37.5	61.0	62.5

For both pre-training metrics (perplexity) and downstream zero-shot evaluations, Mamba outperforms and matches baselines at twice the model size respectively.

MAMBA: Empirical Results

DNA Modelling:

DNA requires LRD and consists of discrete tokens with finite vocabulary, so very similar to Language.



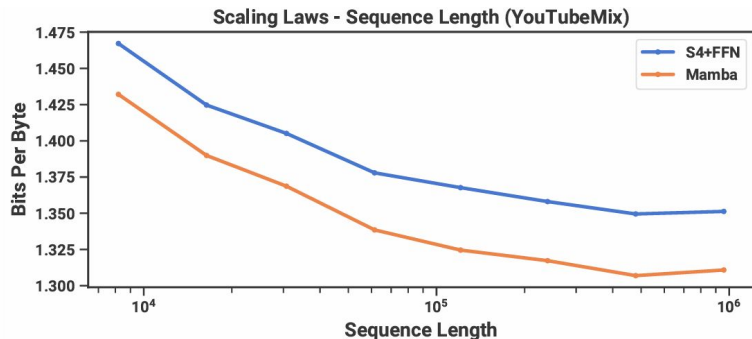
Pretraining perplexity improves consistently as model size or context length increases (holding the other fixed), with Mamba scaling better than HyenaDNA and Transformer++.

MODEL	PARAMS	ACCURACY (%) AT SEQUENCE LENGTH					
		2^{10}	2^{12}	2^{14}	2^{16}	2^{18}	2^{20}
HyenaDNA	1.4M	28.04	28.43	41.17	42.22	31.10	54.87
Mamba	1.4M	31.47	27.50	27.66	40.72	42.41	71.67
Mamba	7M	30.00	29.01	31.48	43.73	56.60	81.31

Downstream Evaluation Task: Great Ape DNA classification

MAMBA: Empirical Results

Audio Modelling and Generation :



Audio Pre-Training: Both Mamba (better) and SaShiMi (U-net backbone with S4+MLP stages) improve consistently with longer context lengths

$$\text{BPB} = \log_2(\text{Negative log-likelihood})$$

MODEL	PARAMS	NLL ↓	FID ↓	IS ↑	MIS ↑	AM ↓
SampleRNN	35.0M	2.042	8.96	1.71	3.02	1.76
WaveNet	4.2M	1.925	5.08	2.27	5.80	1.47
SaShiMi	5.8M	1.873	1.99	5.13	42.57	0.74
WaveGAN	19.1M	-	2.03	4.90	36.10	0.80
DiffWave	24.1M	-	1.92	5.26	51.21	0.68
+ SaShiMi	23.0M	-	1.42	5.94	69.17	0.59
Mamba	6.1M	1.852	<u>0.94</u>	<u>6.26</u>	<u>88.54</u>	<u>0.52</u>
Mamba	24.3M	<u>1.860</u>	0.67	7.33	144.9	0.36
Train	-	-	0.00	8.56	292.5	0.16
Test	-	-	0.02	8.33	257.6	0.19

Auto-regressive Speech Generation: A small Mamba model outperforms the SOTA (and much larger) GAN and diffusion- based models.

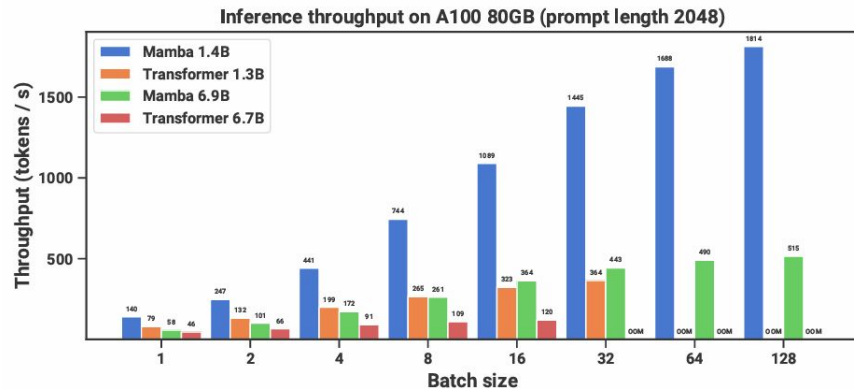
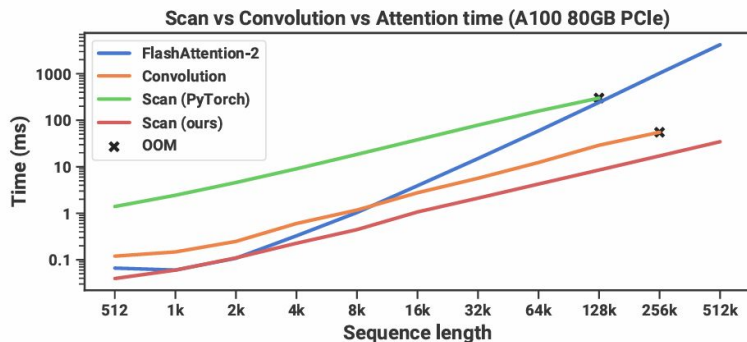
A larger model parameter-matched to the baselines further improves on fidelity metrics.

Tested on **SCo9 dataset**

MAMBA: Empirical Results

Speed and Memory Benchmarks:

Training: Our parallelized selective scan is faster than the best attention implementation (Flash Attention-2) and upto 20-40 $\times$ faster than a standard scan implementation in Pytorch.



Inference: Mamba achieves 4-5 $\times$ higher throughput than a Transformer of similar size, since without the KVcache it can use much higher batch sizes.



My barebones MAMBA
implementation

Conclusion

- SSMs have emerged as an improved sequence model (underlying state space dynamics) over traditional attention-based transformers.
- Use of HiPPO framework for continuous-time memorization was one of the key factors in the empirical success of SSMs, especially in long-range settings.
- SSMs are computationally more efficient and have better performance in most settings.

Further Work

- Analyze privacy leakage in SSMs from LSSL, S4 to MAMBA.
- Integrate privacy amplification by iteration concept into the SSM dynamics to privatize the model for a streaming setting of N inputs, K private followed by $N-K$ public tokens.